Use of Reinforcement Learning to train more realistic, intelligent NPC's and adaptive to real-time changes (2022)

Rafael Ribeiro nº18810

***Abstract*—This article presents an approach of reinforcement learning on the Unity game engine using MLAgents for purposes of training AI in complex escape scenarios. The aim is to develop these agents to the point of being capable of navigating a top-down 2D environment while avoiding the guard's field of view and finding their way out to the escape point. This proposed solution combines PPO (Proximal Policy Optimization) algorithm with the default deep neural network of Unity MLAgents, a feed-forward deep artificial neural network (FF-ANN).**

These agents perceive the environment around through a ray perception sensor, which provides sensorial inputs to the FF-ANN, this sensor data is processed by it allowing the agents to perform random decisions on how to navigate the environment. The PPO is used to optimize the agents policies and adapt their behaviour over time. PPO strikes a balance between ease of implementation, sample complexity, and ease of tuning, trying to compute an update at each step that minimizes the cost function while ensuring the deviation from the previous policy is relatively small. [1].

This combination allows the agents to navigate complex environments and improve their navigation skills and adapting to real-time changes and obstacles. The experimental results demonstrate the effectiveness of the proposed approach of agents adapting to real-time lines-of-sight showcasing the potential of reinforcement learning in training NPC's in complex gaming scenarios.

***Index Terms*— Reinforcement learning, Proximal Policy Optimization, NPCs, ray perception sensor, complex environments, Line-of-sight.**

I. INTRODUCTION

A. Motivation

This project was inspired by the enemies NPC's behavior of F.E.A.R. In which when aware of the player, they approach and attack the player through alternative paths out of the player's line of sight. I was really interested in implementing a much simpler version of this behavior in the context of an agent trying to reach a goal avoiding the player's line of sight.

B. Framing

Everybody loves games. Everybody loves to feel immersed. One of the main problems since the beginning of the creation of games, was exactly that. How to keep our players immersed? In various and various ways, the evolution of games started to achieve a closer look to immersion. Sure, we are not in a time of fully-immersive games, but comparing now to the beginning it is a great improvement. One of the aspects of immersion is how NPC's behave, like movement, decisions, etc… Humans like to be around something they know about, and the opposite to the unknown. So, the closer a NPC behave like a human, higher the immersion the player will feel. Looking at the game flow of games, players also don't like to feel bored/frustrated with the games they're playing. This is something that can become easily obtained if the NPC's don't seem smart, or they don't adapt to the player's skill. NPC's need to be realistic, adaptive and intelligent in order to players feel more immersed, challenged.

C. Objectives

My objective with this project was to train NPC's in complex scenarios to be able to showcase intelligent behaviors, almost human-like, reaching a goal avoiding obstacles.

This project also explores techniques/algorithms of reinforcement learning such as PPO applied to the development of NPC's in complex scenarios, which is something I'm also very interested in studying. With this said, this project is really important in the exploration and creating a path towards that dream of human-like NPC's.

C. Article's Structure

This article is divided into 5 main parts. The first is to introduce the project, discuss the inspiration behind it, the problems I want to address, and the objectives of said project.
Second part is to explain fundamental concepts of machine learning and in my case Reinforcement Learning, its methods, algorithms and its equations relevant to my project.
Third part is to describe the requirements and specifications of my project, for example, environments, Agents, mechanics, the methods, and the solutions adopted.
Fourth part is to present the results of my experiments and evaluations, showing the performances of the NPC's behaviors.
Fifth and final part is to summarize the findings, results and conclusions of the project, emphasizing achievements, difficulties and discuss potential areas of improvement making the project richer.

II. THEORICAL CONCEPTS

- Machine Learning

Machine learning is a field of Artificial Intelligence that allows computers to learn from data, identifying patterns and make predictions/decisions based on this data. There are three subfields of machine Learning:

- Supervised Learning
- Unsupervised Learning
- Reinforcement Learning

In this article I'll only tackle the concepts and equations of what I used in the project.

- Reinforcement Learning

Reinforcement Learning is an area of machine learning tasked on how the agents take certain actions accordingly to their environment around in order to maximize a cumulative reward. [2]. Computers learn to execute actions accordingly to observed data using algorithms, in the case of Unity MLAgents,there are two default available, Proximal Policy Optimization (PPO) and Soft Actor-Critic (SAC).

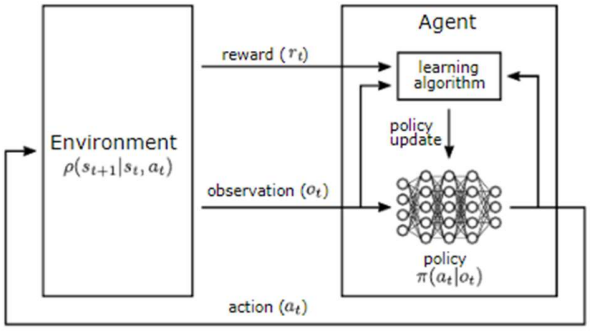

Fig. 1 - Interaction between agent and environment

In the context of the project an agent is in an environment learning it for a finite ammount of steps. In each step *t* the agent is present in a state of the environment s_t, observations of the environment will be made o_t and the policy will process the data and take an action a_t putting the environment in the next state s_{t+1} in which the agent will be received a reward (positive, negative) from the environment. The agent takes in consideration these rewards, processing and learning from them, updating its' own policy with the goal in mind of maximizing the maximum reward possible in the current environment the agent is placed in.

In summary, the agent will be placed in an environment, observing it, taking decisions based on the policy executing said actions to get a reward and updating the policy to reach a maximum reward possible in a finite number of steps. Learning the best policy to achieve that objective is a process of trial-and-error.

In Unity MLAgents there are two phases of RL: A training phase, in which the agent learns the optimal policy through trials, and an Inference phase, in which applies this model to a new and unprocessed data, basically placing the agent in an unknown environment with the optimal policy he got from the training phase.

- Artificial Neural Network

ANN are a subset of machine learning and basically the foundation of deep learning algorithms [3]. They model systems throughout connections of layers that simulate the nervous system of a human.

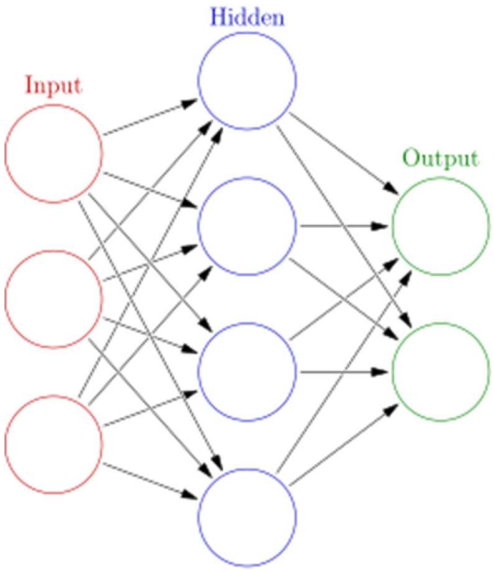

Fig. 2 - Model of a Artificial Neural Network

An ANN contains node layers, the first is the input layer, this is the entry point of data in the system. Then there is at least (can be multiple) one hidden layer where the information is processed, and the output layer where the system processes how to proceed based on this data. Each node has a designated weight and threshold, if the output exceeds a certain threshold it activates passing the processed value to the next layer, this sequential propagation is called feedforward.

- Proximal Policy Optimization

The model used to train in these project was chosen to be the PPO, to talk about PPO first we have to talk about Policy Gradient Methods

Policy gradient methods work by computing an estimator of the policy gradient and plugging it into a stochastic gradient ascent algorithm. The most commonly used gradient estimator has the form [1]

$$\hat{g} = \hat{E}_t [\nabla_\theta \log \pi_\theta(a_t \mid s_t)\hat{A}_t]$$

where π_θ is a stochastic policy and $\hat{A}_t$ is an estimator of the advantage function at timestep t. Here, the expectation $\hat{E}_t [\ldots]$ indicates the empirical average over a finite batch of samples, in an algorithm that alternates between sampling and optimization. Implementations that use automatic differentiation software work by constructing an objective function whose gradient is the policy gradient estimator; the estimator ˆg is obtained by differentiating the objective.

$$L^{PG} = \hat{E}_t [\log \pi_\theta(a_t \mid s_t)\hat{A}_t]$$

While it is appealing to perform multiple steps of optimization on this loss L^{PG} using the same trajectory, doing so is not well-justified, and empirically it often leads to destructively large policy updates and are whore than the "no clipping or penalty" setting.

Proximal policy optimization [4] is a family of policy optimization methods that use multiple epochs of stochastic gradient ascent to perform each policy update. These methods have the stability and reliability of trust-region methods but are much simpler to implement, requiring only few lines of code change to a vanilla policy gradient implementation, applicable in more general settings (for example, when using a joint architecture for the policy and value function), and have better overall performance.

.III. PROJECT DESCRIPTION

- Requirements

Unity MLAgents Toolkit is an open-source project that enables games and simulations to serve as environments for training AI agents, like mentioned before these agents can be trained with three methods. Reinforcement learning (the one I'm using), imitation learning, neuroevolution, or other machine learning methods through a simple to use Python API.

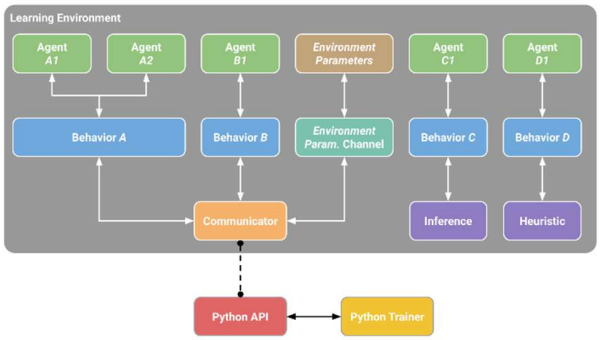

Fig. 3 - Pipeline of the MLAgents Toolkit

This toolkit contains 4 main components:

- *A learning environment*, this is the Unity game scene, which provides an environment for the agents to observe, act and learn. The developer can build this environment depending on their taste and likes, it's recommended that the first environments should be simple, and then when the agent proves themselves to be learning effectively the difficulty of said environment can be amped up.

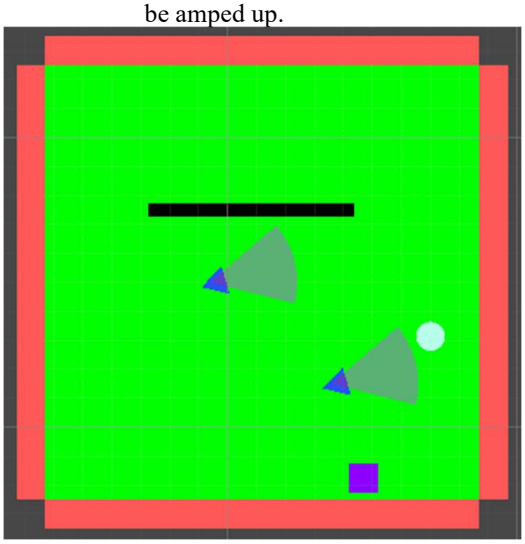

Fig. 4 - Final Environment (Every position random + cops rotation)

In my case, in the first environment I started with my agent (cyan circle), and my end goal (purple square), the second environment I introduced it to randomness of the end goal, the third environment I introduced it to obstacles such as the wall (black rectangle), fourth environment I introduced it to the moving objects (cop) that the agent has to evade, and in the final environment I introduced it to a much bigger wall, two cops with every position random (on a range). I'll discuss rewards and results later.

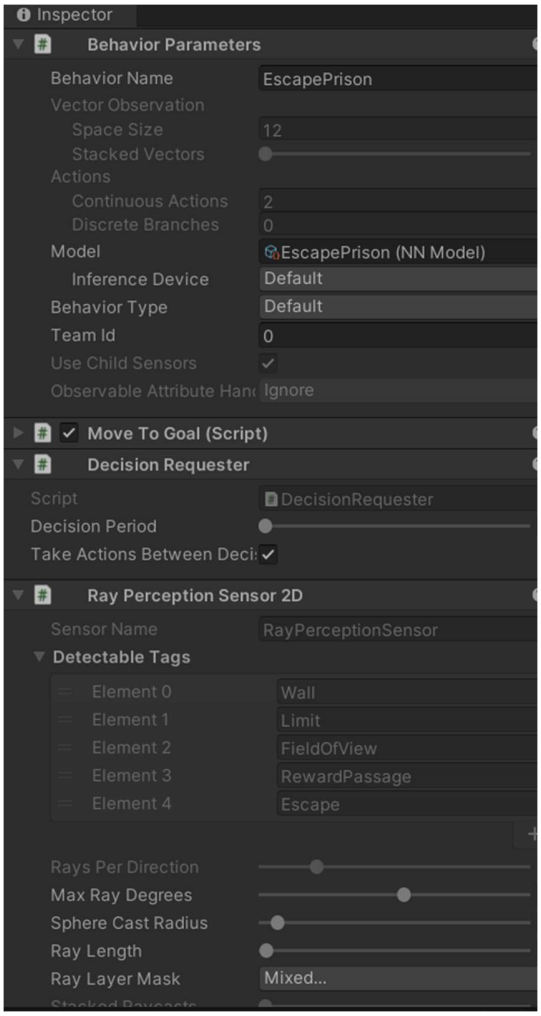

Fig. 5 - Components parameters of the Agent/Behaviour

This learning environment contains two main components, the *agents* that are attached to a Unity GameObject within the scene. This handles all the observations, performing the actions it gets and assign a reward wether positive or negative when the time comes. Each agent is linked to the second key-component, the *Behaviour*, this defines the attributes of the agent like the number of actions the agent can make, it receives observations and rewards and will return the actions. The behaviour has three types, a *training*, which means it's not yet defined but about to be trained and learn from it, after it is defined it becomes an Inference behaviour and can be used as a model for future trainments as a Neural Network file. Then there is a *heuristic behaviour* which is defined by the hard-coded set of code that is inside the script, and then the *Inference behaviour*, that uses the Neural Network file as its' model.

- The second main component is the **Python API** which contains a low level Python interface that is not a part of Unity but connects to it through a communicator to be interacted with and handle the trainings of said learning environment
- The third component is the **external communicator**, it lives inside the learning environment and connects it to the Python API.
- The fourth main-component is the **Python Trainer** that uses all the machine learning algorithms, they are incorporated into the *mlagents* package and expose the *mlagents-learn* utility to be able to use the training methods. This component can only communicate with the Python API.

- Adopted Solution

To train my agent through reinforcement learning I had to setup the agent's actions, observations and rewards.

- Actions

This is a 2D Top down simple-escape game, so the only actions would be moving in the horizontal (left, right), and moving in the vertical (up, down). I opted for using 2 continuous actions as shown in Fig. 5. These actions are floats ranged from [-1, 1] and I could add it to the position of the transform, allowing the agent to move in those directions.

- Observations

For the observations, I knew for sure that the most important ones would be the agent's position, the goal's position, the FOV's position and direction. Because vector observation is an array that takes floats, I had to give it the number of vectors multiplied for the number of floats, 4*3 = 12. [Fig. 5.]

- Decisions

Because this is a complex scenario, in which the agent must make decisions on the spot to not get caught by the cop's line-of-sight, the decision request was set to be asked in every single step. [Fig. 5]

- Rewards

For the rewards my initial process was the agent getting rewarded for successfully passing through the sides of walls and rewarded for reaching the goal. If the agent touched the limits of the board, collided/kept colliding with the wall, or get caught by the cops line-of-sight, then the agent would be punished. After meddling with the values I reached the conclusion that every action that would end the episode should have the same value. For successfully escaping the agent would be rewarded a value of 5, while touching the limit of the environment, or gets caught by the cop line-of-sight was punished with a value of -5. For touching the wall the agent would be punished with a value of -1, with a constant punishment of -0.1 in case the agent kept touching it, to reinforce the agents they're not supposed to be touching it.

- Environment perception

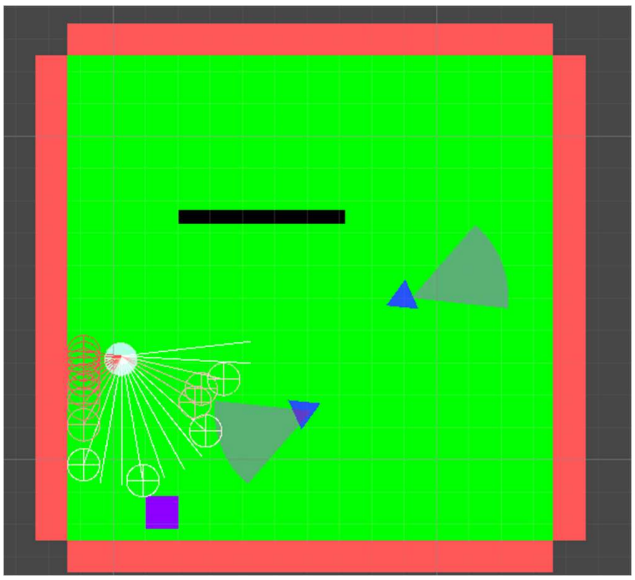

Fig. 6 - Agent raycasts observing the environment

For better interaction and adaptability with the environment around a component *RayPercetionSensor2D* of observation was added to the agent. This component is some kind of sensor that allows the agent to perceive and observe its environment by emitting rays in all kinds of direction, detecting all kinds of objects that those rays hit.

On contrary to *Camera Sensors (perceive visual information)*, all these rays only processes distance, tags, and detected layers. In my project I used every single piece of information that the agent would need to take extra care on, such as the limits, walls, the line-of-sight (field of view), and the escape goal. This was an important component for the agent to move around walls, but also to react better to the cops line-of-sight, which are a moving and rotating target that the agent needs to be more aware of.

For the values used on these I made sure to emit the rays on an angle superior to 90 degrees, for the agent to be able to detect anything approaching from the side and adapt accordingly. The rays length was also made sure to be lengthier than the cop's field of view, just in case it was approached by them in a unfavorable angle.

One positive side of using Unity for this case was the ability to create various instances of the same environment (agents, obstacles, goals), this would allow for a greater number of samples processed by the training buffer, which updates the policy and increases the training performance.

```yaml
behaviors:
  EscapePrison:
    trainer_type: ppo
    hyperparameters:
      batch_size: 512
      buffer_size: 10240
      learning_rate: 1.0e-4
      beta: 5.0e-4
      epsilon: 0.2
      lambd: 0.99
      num_epoch: 3
      learning_rate_schedule: linear
      beta_schedule: constant
      epsilon_schedule: linear
    network_settings:
      normalize: false
      hidden_units: 128
      num_layers: 3
    reward_signals:
      extrinsic:
        gamma: 0.99
        strength: 1.0
    max_steps: 1000000
    time_horizon: 64
    summary_freq: 10000
```

Fig. 7 - Hyperparameters of the config file

After the agent is trained with the training methods from the Python Trainer, this agent gathers the samples/experiences necessary in which is processed by the neural network and optimize the policy and decide which actions the agent has to perform. To achieve a successful training, the hyperparameter of the yaml (config) file need to be updated accordingly to the complexity of the problem. After various simulations here are some information about the parameters used that brought the best results:

- trainer_type: Which trainer to use, there's Proximal Policy Optimization , which when researching articles and comparing it seems consensual it is the best one for my kind of project, then there's soft actor critic and POCA.
- batch_size: the number of experiences in each iteration of the gradiend descent, it has to be multiple times smaller than buffer_size. Has the minimum default value of 512.
- Buffer_size: the number of experiences to be collected before updating the policy model, it has to be multiple times larger than batch_size. Default of 10240.

- Learning_rate: Initial learning rate for gradient descend, it corresponds to the strength of each gradient descend update step. If the training is unstable and rewards don't increase, it should be decreased. Started with 3.0e-4 and updated it to 1.0e-4.
- Beta: Is the strength of the entropy regularization, a higher value makes the policy more random. Default is 5.0e-3
- Epsilon: Influences how quickly the policy can evolve during training. Default is 0.2
- Lambd: Is the regularization parameter used to calculate the Generalized Advantage Estimate, it is how much the agent relies on it's current value estimate when calculating an updated value estimate. Started at 0.95 but ended up at 0.99.
- Num_epoch: Number of passes to make through the experience buffer when performing gradient descent optimization. The default was 3.
- Hidden_units: Number of total units are in each fully connected hidden layers of the neural network. Default at 128
- Num_layers: The number of hidden layers in the neural network. Started at 2, but then I updated it to 3 due to the complexity.
- Extrinsic_gamma: Discount factor for future rewards coming from the environment. Default at 0.99.
- Extrinsic_strength: Factor by which to multiply the reward given by the environment. Default at 1.
- Max_steps: Number of steps that must be taken across all environments before ending the training process. It ranged between 500000-2000000
- Time_horizon: How many steps of experience to collect per-agent before adding it to the experience buffer. Default at 64.
- Summary_freq: Number of experiences that needs to be collected before generating and displaying training statistics. Default 10000.

IV. RESULTS

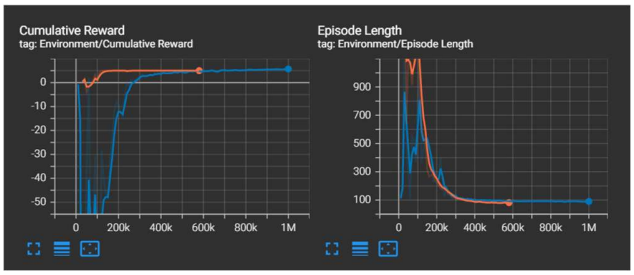

Fig. 8 – First & Second Environment rewards (Agent, Goal, Limits, Wall)

Like mentioned before for the training of the agent I had to start with a simple environment for the agent to learn, this contained the agent of course and the escape goal. With no model to be used the agent had no knowledge of the rewards of said actions to be made, so it made random actions at first. When it started to learn what reward those actions would bring it achieved a maximum mean of cumulative reward of 5 with 100% efficacy, (orange in Fig. 8), which means in every single case the agent was able to find the position of the escape goal, both the agent, and goal had random positions in every step. It's noted that the episode length was high in the beginning, showcasing that the agents had trouble finding their own way to the goal, but decreased when they started to learn the correct actions to be done that maximizes the reward.

After being trained I used this new model as a brain for the environment with the wall. When the wall in random positions was introduced, I added a reward of 1 if the agent passed through the sides of it, which means the maximum reward is now 6. Accordingly to the blue line of Fig.8 the agents had some trouble adapting to this newly-introduced complexity, getting negative rewards for a while, after 800k steps the cumulative reward stagnated at 5.5 (92% of efficacy) with some low variations, which is a pretty good result considered the random values of said wall and agents, looking at the episode length it was noticed that it took just a close bit more steps for the agent to find the goal, but then it adapted.

As a side note, the wall had 2 points of reward (1 point) to the side, to force the agent to pursue those points. Because I focused on the exploitation part more than the exploration, and the observation vectors were only at more than a bit of 90 degrees, the agent would only go to one of those rewards and then to the end goal.

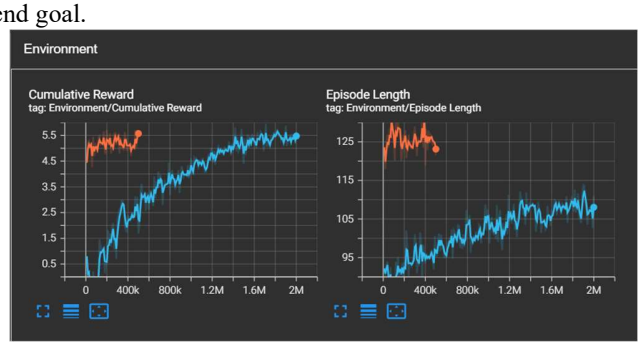

Fig. 9 - Third & Fourth Environment (Cop Static & Moving)

After it learned pretty decently the first two environments, I introduced the agent to the concept of cop. When I introduced a static cop (for simplicity) guarding a side of the wall (cyan line in Fig 9.) it's noted in the cumulative rewards that the agents had a lot of trouble adapting to this newly introduced complexity, it was a slow process but once again it showed some variations around 5.4 (90% of efficacy), with tendency to improve slowly with more training time. One thing to note here is the episode length. This one increased over the number of steps because early on the agents would get caught by the cop's FOV, but due to learning they would survive more time, stagnating to a value a bit higher than the earlier environments because of the new obstacles.

After this I introduced the agent to the fundamental base of this program, one cop moving, which means the agent would need to be aware of the cop's FOV position and direction not only on the beginning of the episode but during the step of the

environment and adapt with it. With the model that the agent trained when the cop was static it noticeable that the initial reward of the agent was already high (orange line in Fig.8), but its improvement was rising but in a slow rate, it appeared to be learning to some extent, but in some cases it would be cornered in unfavorable positions and caught, or even sacrifice a bit of reward touching the wall to not get caught by the cop, getting at some values around 5.2 (86,6% efficacy). With this slow tendency in improving and a bit of instability I decided to regress a bit on the complexity of the level, a bit of a smaller wall, and reduced a bit the speed of the cop, it was at this point I reduced the learning rate to 1.0e-4.

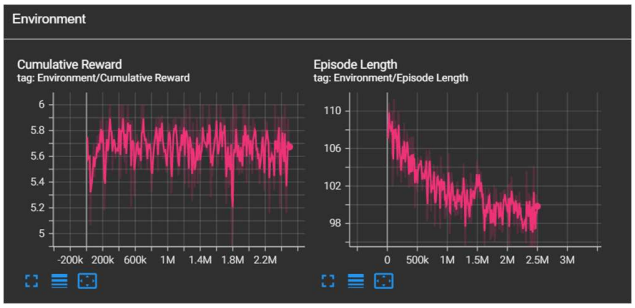

Fig. 10 - Environment Cop Moving - Reduced COmplexity

After reducing the complexity and changing the hyperparameters it was noticeable the improvement of the agent had, reaching to values around 5.7 (95% of efficacy), which are almost excellent results, based on the random position of the obstacles and the unpredictability of the moving FOV. Now with the knowledge of the cop moving, the episode length decreases over time, which showcases the ability of the agent of discovering the best path to the escape goal while avoiding the cop's FOV.

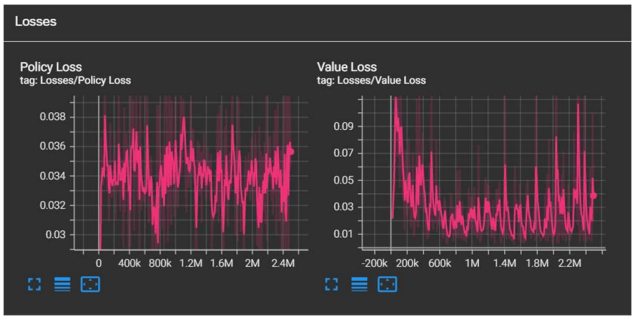

Fig. 11 - Losses of final environment mentioned earlier

The Policy Loss shows the averages value of the policy loss function that decided what actions the agent should execute, this being a value of almost 0 indicates that the agent learned what to do in this environment. The value loss indicates how well the model is able to predict the value of each state. This should increase while the agent is learning and then decrease when the reward stabilizes. [2]

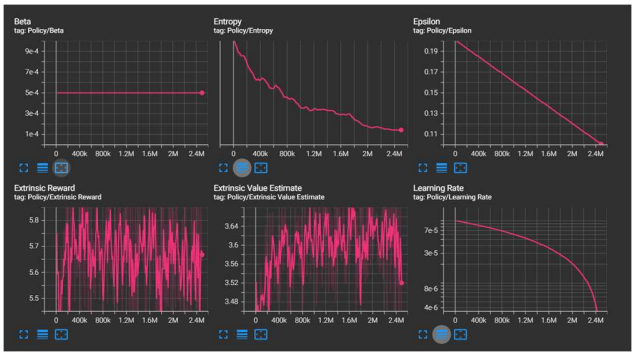

Fig. 12 - Policy of the same final environment

The entropy showcases how random the decisions of the model are, it should slowly decrease during successful training processes.
The extrinsic Reward showcases the mean cumulative reward received from the environment per-episode.
The extrinsic value estimate showcases the mean value estimate for all states visited by the agent, it should increase during successful training sessions. In this case the agent had already knowledge of this scenario and better adapted to the reduced difficulty of it. Here are the status of the Losses and Policy of the earlier environments, showcasing the good adaptability of the environment:

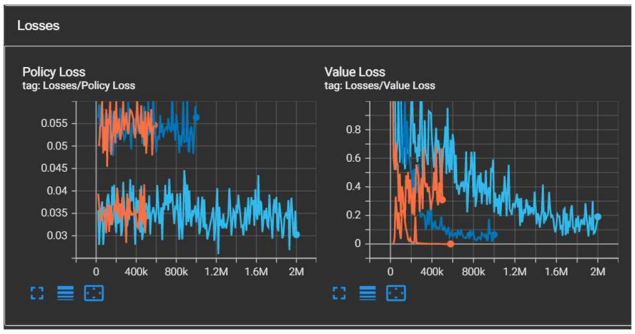

Fig. 13 - Losses of Earlier environments

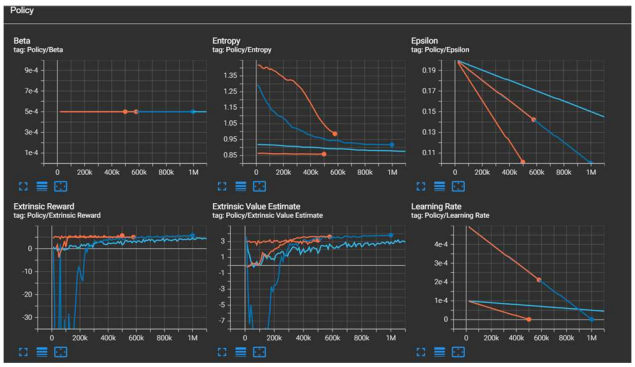

Fig. 14 - Policy of earlier environments

In this case we can see that all the Losses and Policy go by the rule of how successful trainings can go, which some roughness and variability when it came to the first scenario which had no

knowledge at all and when the cop's field of view got introduced.

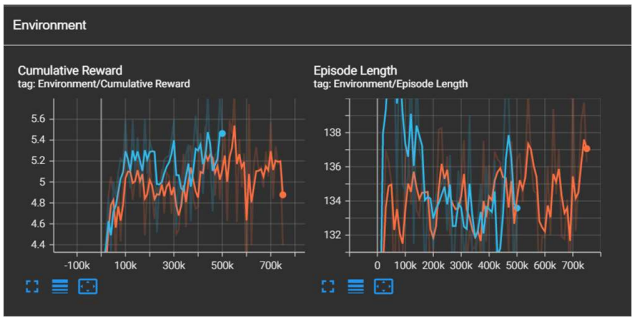

Fig. 15 - Environment with 2 cops

These results being satisfactory I decided to place the agent in a new more complex environment where 2 cops are patrolling near the escape goal, with every position random, and with a constant rotation of the cop's FOV. The results were surprising, now with everything unpredictable, and with two constant moving targets the agent had to be extra aware of what to do. The cyan line represents the two cops without the rotating FOV, the cumulative reward for both started low and then the agent learned about this newly introduced threat, and adapted great to it, achieving a medium of 5.2 (86% efficiency) a bit higher than what I was expecting (around 75%). The episode length also started low cause the agent kept getting caught by the second FOV, then it became high when trying to advert said FOV, and decreased stabilizing when it learned how to successfully move around both cops' FOV and arriving at the escape goal. With this good but unstable results I changed back the learning rate.

The orange line represent the environment where both cops' had now a rotating FOV, having a similar but easier learning of the same environment, but with these new unpredictability of the moving and rotating FOV it only achieved a mean cumulative reward of 5/5.1 (83-85% efficacy) which also surpassed the expectations of an efficacy of 70%.

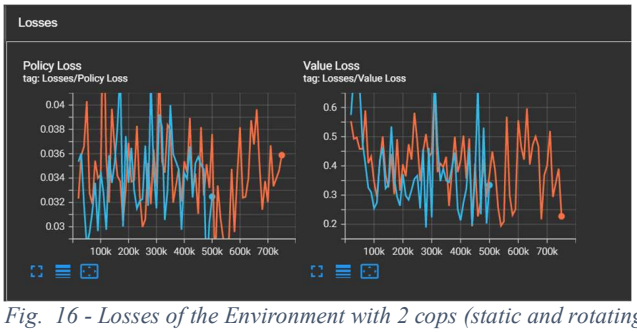

Fig. 16 - Losses of the Environment with 2 cops (static and rotating)

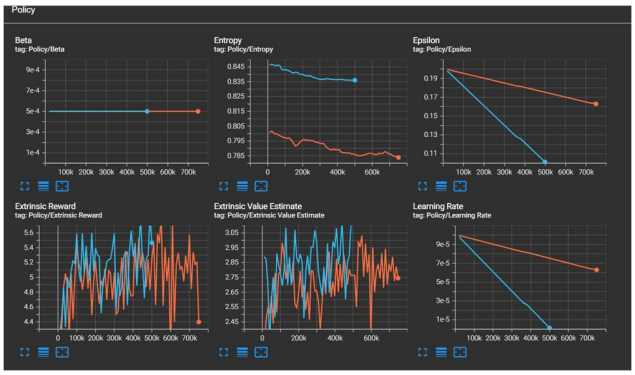

Fig. 17 - Policy of the Environment with 2 cops (static and rotating)

These losses and policy are a bit more unstable than the earlier ones presented, it's due to the higher complexity presented to the agents, but it still follows to some extent the rules of a good successful training.

V. CONCLUSION AND FUTURE WORKS

The conclusion I arrive with these results is, that in fact, reinforcement learning can be a great way of improving NPC's to be more realistic, intelligent and adaptive to real-time moving targets, I agree that the results were not perfect due to the unpredictability of the environment, but they have shown the promising future of this kind of implementation. With future works the results can have greater efficacy and the agents a better and faster learning system with the implementation of Camera Sensors and the use of Convoluted Neural Networks to perceive visual information on real-time of the guard's FOV position and direction, like it was suggested by many articles of these kind of visual-perception of moving objects in real time.

VI. REFERENCES

ACKNOWLEDGMENT

I would like to thank my professor of teaching us the various topics and methods of Artificial Intelligence from Neural Networks to Pathfind with easy explanations and the practical application of those methods.

I would also like to thank Queirós, Nova and Nuno, my friends, for giving me moral support during the development of this project and article.

REFERENCES

[1] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, Oleg Klimov, "Proximal Policy Optimization Algorithms" arXiv:1707.06347, 20 Jul 2017

[2] Unity Technologies. Background: Machine Learning https://github.com/Unity-Technologies/ml-agents/blob/develop/docs/Background-Machine-Learning.md 06/2018

[3] IBM. What is a neural network? https://www.ibm.com/topics/neural-networks

[4] .Unity Technologies. Unity TensorBoard to observe training. https://unity-technologies.github.io/ml-agents/Using-Tensorboard/

[5] R. J. Hijmans and J. van Etten, "Raster: Geographic analysis and modeling with raster data," R Package Version 2.0-12, Jan. 12, 2012. [Online]. Available: http://CRAN.R-project.org/package=raster

Rafael Faria – Bachelor in “Digital Game Development Engineering”. Currently pursuing a Masters’ degree in Digital Game Development Engineering”